

多 Agent 旅遊規劃系統

系統架構設計書 v3

第七組

一、系統概述

本系統為一套多 Agent 旅遊規劃系統。使用者輸入旅遊需求（目的地、天數、預算、興趣偏好、旅遊風格），系統由五個專職 AI Agent 協同合作，自動產出完整且經過合理性驗證的旅遊行程規劃。

系統採用以下技術組合：

- Azure OpenAI：透過 Microsoft Azure 雲端服務呼叫 GPT 語言模型，提供高品質的自然語言推理能力
- LM Studio：作為 Azure OpenAI 的本地端操作介面，透過設定 Azure API 金鑰與端點進行連線
- CrewAI：多 Agent 協作框架，負責統籌各 Agent 的任務分配與執行流程
- Google Maps API：提供三項核心功能
 - Places API (Text Search)：將 LLM 輸出的景點名稱轉換為確定的 place_id 與座標 (Grounding)
 - Distance Matrix API：計算景點間的實際移動時間與距離
 - Maps JavaScript API：於前端介面進行路線視覺化
- Streamlit：用於建構使用者操作介面，包含旅遊需求輸入表單與地圖展示
- Langfuse：AI Observability 工具，記錄每個 Agent 的處理過程、耗時、Token 用量與 Grounding 成功率

二、五大 Agent 架構

系統由以下五個 Agent 組成，各司其職。其中行程 Agent 與交通 Agent 採「協作式計畫」設計，在行程生成階段即進行移動成本最佳化，而非事後補救。

Agent 名稱	職責說明	輸出內容
行程 Agent	根據使用者的旅遊風格與天數，以同一地理區域內的景點為原則，每日產出 2~3 個路線候補方案。接收跨日狀態上下文（前日終點、剩餘預算、已訪景點清單）作為起始條件，並依此約束候補方案的地理範圍與景點選擇	每日 2~3 個候補方案（JSON 格式，含景點名稱清單）
交通 Agent	接收行程 Agent 的候補方案，執行兩階段	確定行程 + 各景點

	處理：(1) Grounding：呼叫 Places API 將景點名稱轉換為 place_id；(2) 最佳化選擇：呼叫 Distance Matrix API 計算各候補的合計移動時間，綜合考量時間效率與交通手段合理性（排除極端高費用或不合理交通方式），選出最佳方案。若 Grounding 失敗，立即觸發短路（Short-circuit）例外處理，不進入後續流程	place_id + 移動時間明細；或 Grounding 失敗通知
預算 Agent	根據確定行程計算各項支出（住宿、交通、餐費、門票），確認是否在剩餘預算範圍內，並更新剩餘預算。僅在交通 Agent 成功回傳確定行程後方可執行	費用明細、剩餘預算更新值、超支警示
美食 Agent	依據使用者偏好與當日剩餘餐費預算，推薦各日用餐地點。僅在交通 Agent 成功回傳確定行程後方可執行	用餐地點推薦清單
檢查 Agent	綜合審查當日確定行程，確認整體合理性，並對行程品質進行 1 至 5 分評分。若行程未通過審查，觸發重新規劃（Trigger re-planning）；若同一日已達重試上限（兩次），則標記警告並強制推進至下一日	「審查通過」或「觸發重新規劃：原因說明」+ 品質評分

三、整體流程

本系統以「每日為單位」逐日執行以下循環，直到所有天數完成後彙整輸出完整行程。系統在各關鍵節點設有例外處理機制，確保狀態異常不會向後傳播。

3-1 系統啟動與初始化狀態

①使用者透過 Streamlit 介面輸入需求（目的地、天數、預算、旅遊風格、每晚住宿地點）

②CrewAI Orchestrator 接收需求，初始化跨日狀態（Cross-day State）：

- 出發地點：使用者指定，若未指定則預設為目的地主要車站或市區中心
- 第一日住宿地點：由使用者於輸入表單中提供（例如：已預訂飯店名稱）；若未提供，則以目的地主要車站為預設基準點。後續各日住宿地點以相同方式於行程開始前由使用者確認，系統不負責住宿選址決策
- 總預算：使用者輸入值，幣別由使用者指定
- 已訪景點清單：初始化為空集合

3-2 每日計畫循環（逐日重複執行）

③行程 Agent 接收當日跨日狀態上下文，以前日住宿地點為出發基準，在鄰近地理區域內產出 2~3 個候補路線方案

④交通 Agent 執行兩階段處理：

- Grounding：呼叫 Places API 將各候補中的景點名稱轉換為 place_id 與座標
- 最佳化選擇：呼叫 Distance Matrix API 計算各候補合計移動時間，綜合考量時間效率與交通手段合理性後，選出最佳方案作為當日確定行程

△例外處理（Short-circuit）：若 Grounding 步驟失敗（任一景點無法解析），系統立即執行短路（Short-circuit）處理：跳過預算 Agent 及美食 Agent，直接觸發重新規劃（Trigger re-planning），要求行程 Agent 提供替代景點。空資料不得流入後續流程。若同一日已達重試上限（兩次），則由檢查 Agent 標記警告並強制推進至下一日，相關異常記錄至 Langfuse。

⑤預算 Agent 計算當日費用，確認不超過剩餘預算，並產出更新後的剩餘預算值

⑥美食 Agent 推薦符合當日餐費配額的用餐地點

⑦檢查 Agent 審查當日完整行程並評分：

- 審查通過 → 更新跨日狀態上下文（當日終點、住宿地點、剩餘預算、已訪清單），推進至下一日
- 審查未通過（未達重試上限）→ 觸發重新規劃（Trigger re-planning），回至步驟③
- 審查未通過（已達重試上限）→ 標記警告，強制推進至下一日

3-3 輸出

⑧所有天數完成後，Streamlit 介面輸出完整文字行程，並透過 Google Maps JavaScript API 呈現各日路線地圖

⑨全程由 Langfuse 記錄每個 Agent 的輸入、輸出、耗時、Token 費用、Grounding 成功率及例外處理次數

四、Google Maps API 的應用與 Grounding 設計

4-1 三項 API 的功能分工

API 功能	使用位置	目的
Places API (Text Search)	交通 Agent (Grounding 步驟)	將 LLM 輸出的景點名稱轉換為確定的 place_id 與精確座標，消除 LLM 幻覺造成的錯誤；解析失敗時觸發短路例外處理
Distance Matrix API	交通 Agent (最佳化選擇步驟)	計算候補路線中各景點間的實際移動時間，作為綜合評估最佳方案的數據依據
Maps JavaScript API	Streamlit 介面 (前端)	將確定行程的 place_id 與座標視覺化呈現於地圖，顯示各日路線

4-2 Grounding 步驟的設計、例外處理與短路機制

行程 Agent (LLM) 輸出的景點名稱為自然語言文字，直接傳入 Distance Matrix API 存在以下風險：LLM 可能產生不存在的設施名稱、細微的錯字，或過於口語化的名稱，導致 API 回傳錯誤並使系統停止運作。

為此，本系統在行程 Agent 與交通 Agent 的處理之間，以 Python 函式（非 LLM）實作 Grounding 步驟。此步驟為確定性函式，不消耗 Token，執行速度快，且具備完整的短路例外處理機制：

步驟	處理內容	失敗時的例外處理
Step 1	以「景點名稱 + 城市名」呼叫 Places API Text Search，取得 place_id 與座標	進入 Step 2
Step 2	擴大查詢範圍，加入目的地國家名稱重新查詢	進入 Step 3（短路觸發）
Step 3 （短路）	解析失敗確認。系統立即中斷當日流程，跳過預算 Agent 及美食 Agent，直接觸發重新規劃（Trigger re-planning），要求行程 Agent 提供替代景點	若已達重試上限（兩次），由檢查 Agent 標記警告並強制推進，異常記錄至 Langfuse

此短路設計確保空資料或錯誤資料不會向後流入預算 Agent 及美食 Agent，從根本避免系統崩潰。所有短路事件均被 Langfuse 記錄，可作為評估行程 Agent Prompt 品質的客觀指標。

4-3 交通最佳化的評估基準

交通 Agent 在選擇最佳候補方案時，不以「合計移動時間最短」作為單一評估基準，而採綜合評估機制，避免選出雖移動時間稍短但費用極端或方式不合理的路線。

評估基準如下：

評估維度	說明
時間效率	各候補方案的合計移動時間，作為主要排序依據
交通手段合理性	優先選用大眾運輸（地鐵、公車、火車）；計程車或私車僅在景點間無大眾運輸可達時採用
費用合理性	排除合計交通費用超過當日交通預算上限的候補方案，即使其移動時間較短
綜合選擇原則	在符合交通手段合理性與費用合理性的候補方案中，選出時間效率最佳者作為確定行程

此設計使本系統得以主張「合理的路線規劃」，依據如下：

- 移動時間來自 Google Maps 實際路況資料，而非 LLM 推測

- 景點座標已透過 Grounding 步驟驗證，確保 API 呼叫的正確性
- 行程 Agent 與交通 Agent 在計畫階段即協作最佳化，而非事後修正
- 最佳化基準同時涵蓋時間效率、交通手段合理性與費用合理性，避免極端路線被選出

五、Azure OpenAI 與 LM Studio 的使用方式

本系統使用 Azure OpenAI 作為所有 Agent 的語言模型來源。LM Studio 作為本地端的操作介面，透過在設定中輸入 Azure 的 API 金鑰與端點 URL，即可直接呼叫 Azure 上的 GPT 模型，無需另行啟動本地語言模型。

此架構的優點如下：

- 所有 Agent 使用同一模型來源，輸出品質一致，設定單純
- Azure 提供 100 美元免費額度，對 Demo 規模的使用量而言綽綽有餘
- 無需管理本地端 GPU 資源，避免因硬體差異造成的環境問題
- CrewAI 透過 LiteLLM 連接模型，SKILL.md 與任務定義完全不受模型設定影響

開始前需確認以下三項資訊：

設定項目	說明
Azure 端點 URL	由申請 Azure OpenAI 服務的成員提供
Azure API 金鑰	由申請 Azure OpenAI 服務的成員提供
部署模型名稱	例如：gpt-4o-mini（於 Azure 後台確認）

六、SKILL.md 設計說明

本系統採用課程中介紹的 Skill 概念，為每個 Agent 撰寫一份專屬的 SKILL.md 指示書。SKILL.md 以 Markdown 格式撰寫，定義 Agent 的行為規則、輸出格式與限制條件，是 Super Power 框架中 Skill Library 的具體實作。

每份 SKILL.md 包含以下欄位：

- 角色（Role）：說明此 Agent 的身分定位
- 目標（Goal）：明確定義此 Agent 的任務目的
- 可使用的工具（Tools）：列出此 Agent 可呼叫的外部 API 或函式
- 輸入規格（Input）：明確定義此 Agent 接收的跨日狀態上下文格式
- 動作規則（Rules）：規定 Agent 在不同情境下的處理方式，包含例外處理條件
- 輸出格式（Output Format）：指定回傳資料的格式（通常為 JSON）
- 禁止事項（Restrictions）：明確排除不屬於此 Agent 職責的行為

各 Agent 的關鍵規則舉例：

Agent	SKILL.md 關鍵規則（節錄）
行程 Agent	- 必須接收跨日狀態上下文（前日住宿地點、剩餘預算、已訪景點清單）作為起始條件 - 每日產出 2~3 個候補方案，每個方案盡量集中於同一地理區域 - 禁止在候補方案中重複已訪景點清單中的景點 - 輸出當日最終抵達地點（作為次日跨日狀態上下文的組成部分）
交通 Agent	- 禁止自行估算移動時間，必須透過 Grounding 步驟取得 place_id 後再呼叫 Distance Matrix API - 綜合評估時間效率、交通手段合理性與費用合理性，選出最佳方案；不得以移動時間唯一標準 - 若 Grounding 失敗，立即觸發短路（Short-circuit）例外處理，不得輸出空資料或錯誤資料至後續 Agent
預算 Agent	- 必須接收跨日狀態上下文中的剩餘預算作為計算基準 - 輸出必須包含更新後的剩餘預算值（供次日跨日狀態上下文使用） - 僅在交通 Agent 成功回傳確定行程後方可執行；若收到 Grounding 失敗通知，禁止啟動
檢查 Agent	- 同一日最多允許觸發重新規劃（Trigger re-planning）兩次 - 達到重試上限時，強制推進並標記當日行程為「警告狀態」，異常記錄至 Langfuse - 審查完成後必須輸出 1 至 5 分的品質評分並傳至 Langfuse

七、Langfuse 的應用

Langfuse 作為本系統的 AI Observability 工具，橫跨所有 Agent 與外部 API 呼叫，記錄系統的完整運作狀況，包含正常流程與例外處理事件。

7-1 Trace 追蹤

完整記錄每次規劃請求的執行路徑，包含各 Agent 的輸入與輸出、Grounding 步驟的成功與失敗、短路事件的觸發記錄，以及 Distance Matrix API 的回傳結果。當系統出現問題時，可快速定位異常發生的環節。

7-2 成本與效能監控

即時監控各 Agent 的 Token 消耗與 Azure OpenAI 費用，以及 Google Maps API 的呼叫次數。由於 Grounding 步驟為確定性函式（不消耗 Token），Langfuse 可清楚呈現 LLM 費用與 API 費用的個別佔比，協助識別高成本節點並進行針對性優化。

7-3 Grounding 品質監控與例外處理統計

Langfuse 記錄每次 Grounding 步驟的成功次數、失敗次數與短路觸發次數。若 Grounding 失敗率偏高，代表行程 Agent 的 Prompt 需要調整（例如：要求輸出更規範的景點正式名稱）。此指標提供客觀依據，使 Prompt 的優化方向可量化評估，而非依賴主觀判斷。

7-4 品質評分（LLM Evaluation）

檢查 Agent 在完成每日審查後，對行程品質進行 1 至 5 分的評分並回傳至 Langfuse。此機制對應課程第四堂介紹的 LLM Evaluation 概念，讓每日行程的輸出品質可被量化、跨日比較與持續追蹤，並可作為後續 Prompt 調整的依據。

八、Super Power 概念的應用

本系統參考前次小組作業中第八組所介紹的 Superpowers 框架，將其核心設計思想全面融入系統架構。Superpowers 的精髓在於「不讓 AI 任意行動，而是透過結構化流程約束 AI 的行為」。

Brainstorming 階段

系統在開始規劃前，透過一問一答的方式向使用者確認所有關鍵資訊（目的地、天數、預算幣別、每晚住宿地點、旅遊節奏等）。資訊齊全並經使用者確認後，才進行初始化跨日狀態並進入規劃流程。此設計對應 Superpowers 中「先對齊需求，再執行任務」的原則，避免因資訊不足而在規劃途中產生系統性錯誤。

Atomization 原子化（含跨日狀態傳遞設計）

行程 Agent 不會一次性產出所有天數的行程，而是以「每日」為單位逐步規劃。每完成一日並通過檢查 Agent 審查後，系統將當日的執行結果封裝為跨日狀態上下文（Cross-day State Context），作為下一日規劃的強制初始條件：

跨日狀態項目	資料來源	用途
當日最終抵達地點 (place_id + 座標)	交通 Agent 輸出	確保次日行程從前日終點出發，維持動線的地理連貫性
當日住宿地點 (place_id + 座標)	使用者於系統啟動時提供	作為次日出發的基準點，避免晨間出現不合理的長途移動
剩餘預算（含各類別明細）	預算 Agent 輸出	作為次日預算 Agent 的計算基準，確保全程預算不超支
已訪景點清單	逐日累積	行程 Agent 禁止在後續天數重複推薦已訪景點，避免 LLM 幻覺造成重複規劃

此設計確保每一天的規劃皆以「前一天結束時的系統狀態」為起點，動線連貫性與預算整合性貫穿整個旅程，並可在 Langfuse 中完整追蹤每日的狀態傳遞記錄。

Skill Library

為每個 Agent 撰寫專屬的 SKILL.md 指示書，作為可重複使用的行為定義文件。此對應 Superpowers 框架中 Skill Library 的概念，確保 Agent 的行為穩定可預測，並可在不影響其他 Agent 的情況下獨立調整與優化。

Verification 驗證

每日行程完成後，必須通過檢查 Agent 的審查才能推進至下一日。審查未通過則觸發重新規劃（Trigger re-planning）；同一日達到重試上限（兩次）後，系統強制推進並標記警告，防止無限循環導致系統停滯。

九、總結

本系統架構以三個核心設計原則為基礎：

- 協作式計畫（而非事後修正）：行程 Agent 與交通 Agent 在計畫階段即協作，透過候補方案搭配 Distance Matrix API 的綜合評估，在行程確定前完成移動成本最佳化，大幅減少事後差退的發生機率
- 確定性 Grounding 搭配短路例外處理：所有景點座標皆透過 Places API 驗證；解析失敗時立即觸發短路（Short-circuit），跳過後續 Agent，直接觸發重新規劃，確保錯誤資料不向後傳播，從根本消除系統崩潰風險
- 跨日狀態傳遞（Cross-day State Passing）：每日規劃結束後，以結構化格式明確傳遞住宿地點、終點座標、剩餘預算與已訪景點清單，確保動線連貫性與預算整合性貫穿整個旅程

系統同時整合了課程中介紹的多項核心概念：AI Agent 架構、Tool Use（Google Maps API）、Skill 設計（SKILL.md）、Langfuse Observability、LLM Evaluation，以及 Superpowers 框架的結構化思維。

最終成果將透過 Azure OpenAI 驅動五大 Agent 協同運作，以 Streamlit 介面進行 Demo 展示，搭配 Google Maps 地圖視覺化與 Langfuse 儀表板。本系統不只是「會排行程的 AI」，更是一個具備真實資料驅動路線最佳化、完整例外處理機制、可被觀察量化評估，且能持續改善的智慧旅遊規劃系統。